

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Грищук Елизавета Дмитриевна
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать разделение монолитного REST API приложения бронирования столиков в ресторанах на микросервисы, опираясь на архитектуру, спроектированную в ДЗ4.

Ход работы

На основе технического дизайна из ДЗ4 монолитное приложение было разделено на 4 микросервиса и API Gateway. Стек технологий: Node.js, TypeScript, Express, TypeORM, PostgreSQL, axios.

Реализованы следующие сервисы:

auth-service (порт 3001) — регистрация, авторизация и управление профилем пользователя. База данных: `auth_db`, таблица `users`. Реализованы эндпоинты: `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/me`, `PATCH /api/users/me`. Внутренний эндпоинт для межсервисного взаимодействия: `GET /internal/users/{id}`.

restaurant-service (порт 3002) — управление категориями, ресторанами и столиками. База данных: `restaurant_db`, таблицы `category`, `restaurant`, `table`. Реализованы эндпоинты: `GET /api/categories`, `GET /api/restaurants`, `GET /api/restaurants/{id}`, `GET /api/restaurants/{id}/tables`. Внутренние эндпоинты: `GET /internal/restaurants/{id}`, `GET /internal/tables/{id}`.

booking-service (порт 3003) — создание, просмотр и отмена бронирований. База данных: `booking_db`, таблица `booking`. Реализованы эндпоинты: `GET /api/bookings`, `POST /api/bookings`, `GET /api/bookings/{id}`, `PATCH /api/bookings/{id}/cancel`. При создании бронирования сервис выполняет межсервисные HTTP-запросы к `auth-service` для проверки существования пользователя и к `restaurant-service` для проверки существования столика и его вместимости.

review-service (порт 3004) — создание и просмотр отзывов. База данных: `review_db`, таблица `review`. Реализованы эндпоинты: `GET /api/restaurants/{id}/reviews`, `POST /api/restaurants/{id}/reviews`. При создании отзыва сервис выполняет межсервисные HTTP-запросы к `auth-service` и `restaurant-service` для проверки существования пользователя и ресторана.

api-gateway (порт 3000) — единая точка входа для клиента. Реализован с использованием библиотеки `http-proxy-middleware`. Маршрутизирует входящие запросы к нужному сервису по префиксу пути согласно таблице из ДЗ4.

Каждый микросервис имеет собственную изолированную базу данных PostgreSQL в соответствии с принципом `database-per-service`. Прямые JOIN между базами данных разных сервисов исключены — целостность данных обеспечивается на уровне приложения через межсервисные HTTP-запросы с использованием `axios`.

Все сервисы успешно запущены локально и протестированы через Postman. Основной сценарий — регистрация, авторизация, просмотр ресторанов и создание бронирования — работает корректно через API Gateway.

Вывод

В ходе выполнения лабораторной работы монолитное приложение было успешно разделено на 4 микросервиса и API Gateway в соответствии с архитектурой, спроектированной в ДЗ4. Каждый сервис имеет собственную базу данных. Реализовано межсервисное взаимодействие через синхронные HTTP-запросы. Все 13 эндпоинтов работают корректно через единую точку входа на порту 3000.